

PLPP Assignment 5: Interpreter Implementation

Anna Mohil

Git repo link:

<https://github.com/VY-Assignments/interpreter-anna-mohil.git>

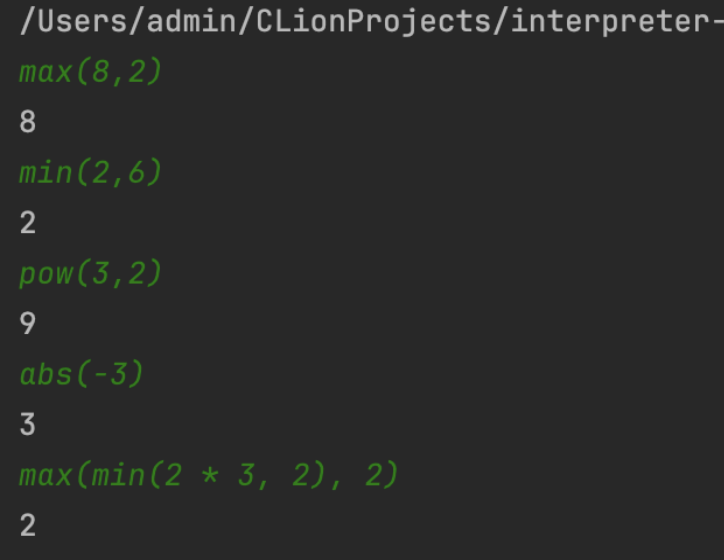
1. Program Explanation

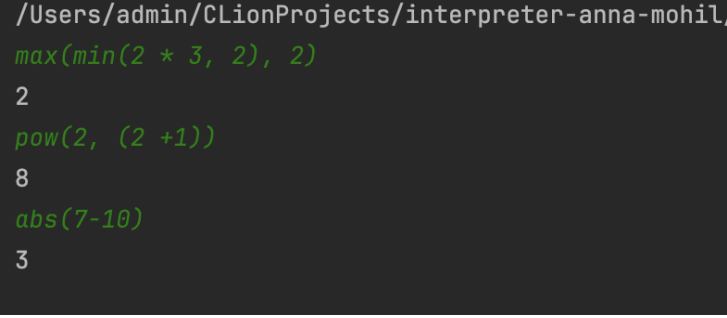
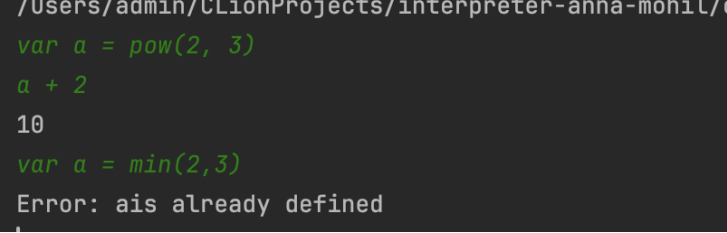
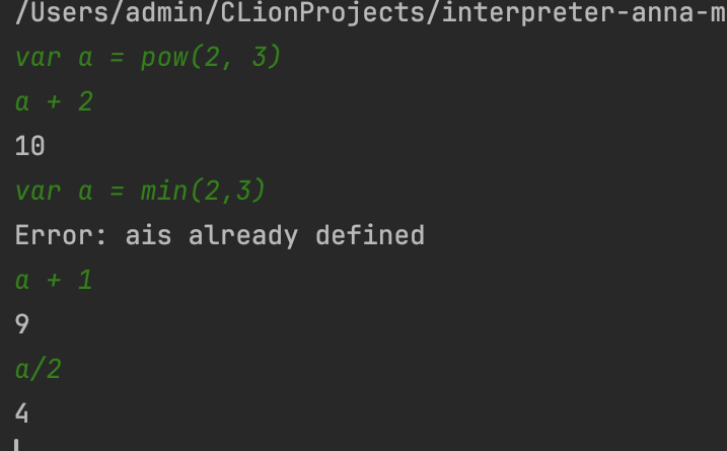
- **Interpreter Architecture:** Describe the processing lifecycle of an expression inside your C++ program (e.g., Lexer/Tokenizer, Parser, Abstract Syntax Tree (AST) creation, and Evaluator).
- **Functional Concepts in C++:** Explain how you implemented and enforced functional immutability inside an imperative/OOP language like C++. Describe how your Environment mapping blocks variable re-assignment.
- **Environment State Management:** Detail how variables (var) and user-defined functions (def) are stored in memory across multiple interactive command lines.
- **Dynamic Parsing & Evaluation:** Explain how nested/combined expressions and higher-order functions (like the numerical integral evaluation) are recursively parsed and evaluated.

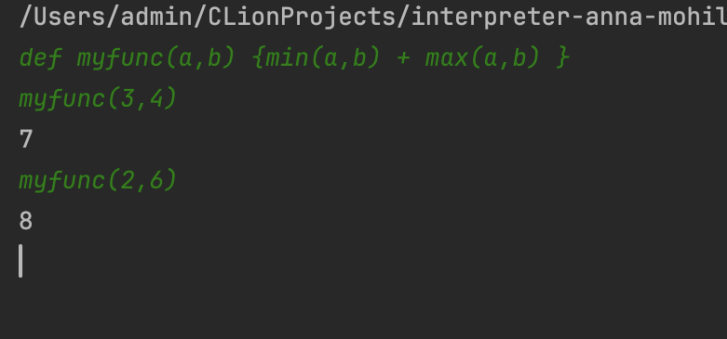
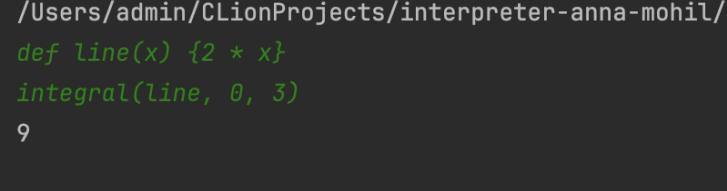
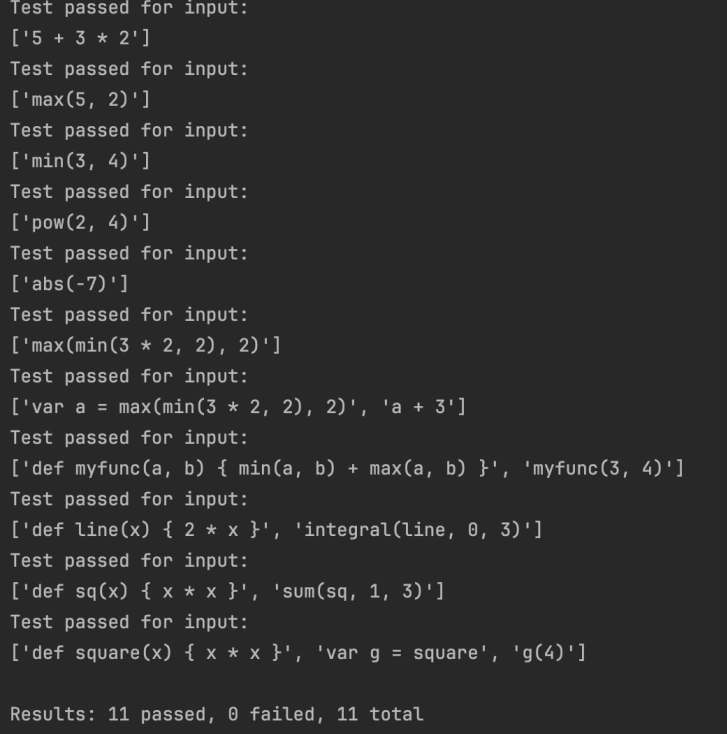
2. Test Plan — Screenshots per scenario

Base Flows

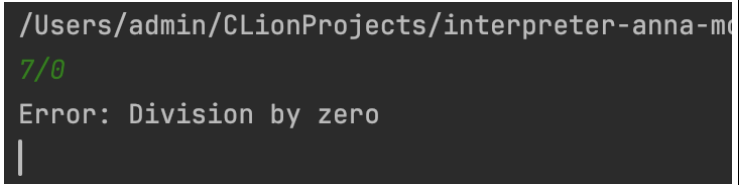
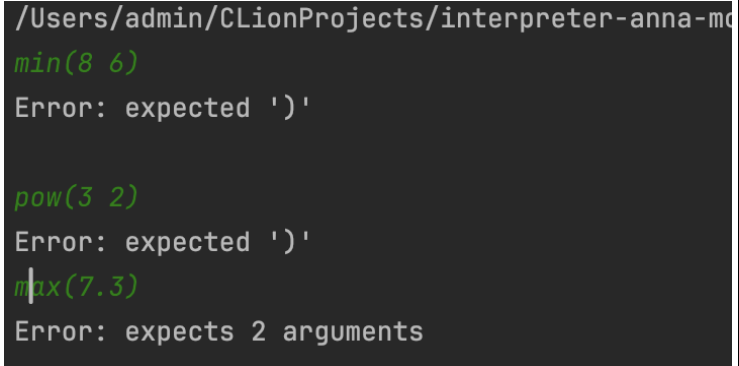
Scenario / Test Case	Expected Result
Basic Arithmetic Operations	Expressions containing +, -, *, / execute correctly according to standard operator precedence (e.g., $5+3*2$ yields 11).

Scenario / Test Case	Expected Result
Screenshot: <PUT HERE>	 <pre> /Users/admin/CLionProjects/interpreter-anna 5+3*2 11 7/7 1 8 + 2 10 3 -1 2 25.2*2 50.4 </pre>
Built-in Function Evaluation	Execution of built-in operations like pow, abs, max, and min yields proper computational outputs.
Screenshot: <PUT HERE>	 <pre> /Users/admin/CLionProjects/interpreter- max(8,2) 8 min(2,6) 2 pow(3,2) 9 abs(-3) 3 max(min(2 * 3, 2), 2) 2 </pre>
Combined & Nested Expressions	Deeply nested combinations evaluate accurately without losing semantic context (e.g., max(min(3*2,2),2) evaluates cleanly to 2).

Scenario / Test Case	Expected Result
Screenshot: <PUT HERE>	 <pre> /Users/admin/CLionProjects/interpreter-anna-mohil/ max(min(2 * 3, 2), 2) 2 pow(2, (2 + 1)) 8 abs(7-10) 3 </pre>
Variable Binding (var)	Declaring an immutable variable using var \$a = ... binds the value to the identifier successfully within the environment state.
Screenshot: <PUT HERE>	 <pre> /Users/admin/CLionProjects/interpreter-anna-mohil/ var a = pow(2, 3) a + 2 10 var a = min(2,3) Error: ais already defined </pre>
Variable Reuse	Referencing a previously declared immutable bound variable in a subsequent expression evaluates correctly (e.g., \$a+3).
Screenshot: <PUT HERE>	 <pre> /Users/admin/CLionProjects/interpreter-anna-m var a = pow(2, 3) a + 2 10 var a = min(2,3) Error: ais already defined a + 1 2 a/2 1 </pre>
User-Defined Functions (def) — Bonus	Defining a custom function (e.g., def myfunc(a, b) { expression }) binds the macro/AST structure so it can be called later like a built-in.

Scenario / Test Case	Expected Result
Screenshot: <PUT HERE>	 <pre> /Users/admin/CLionProjects/interpreter-anna-mohil def myfunc(a,b) {min(a,b) + max(a,b) } myfunc(3,4) 7 myfunc(2,6) 8 </pre>
Higher-Order Integral — Bonus	Evaluating integral(f, a, b) applies the trapezoidal approximation rule successfully over a one-argument function.
Screenshot: <PUT HERE>	 <pre> /Users/admin/CLionProjects/interpreter-anna-mohil/ def line(x) {2 * x} integral(line, 0, 3) 9 </pre>
Python Test Script Harness	Running py.exe .test_script.py spawns the interactive C++ program, streams test datasets through stdin, and prints passing validation hooks.
Screenshot: <PUT HERE>	 <pre> Test passed for input: ['5 + 3 * 2'] Test passed for input: ['max(5, 2)'] Test passed for input: ['min(3, 4)'] Test passed for input: ['pow(2, 4)'] Test passed for input: ['abs(-7)'] Test passed for input: ['max(min(3 * 2, 2), 2)'] Test passed for input: ['var a = max(min(3 * 2, 2), 2)', 'a + 3'] Test passed for input: ['def myfunc(a, b) { min(a, b) + max(a, b) }, 'myfunc(3, 4)'] Test passed for input: ['def line(x) { 2 * x }, 'integral(line, 0, 3)'] Test passed for input: ['def sq(x) { x * x }, 'sum(sq, 1, 3)'] Test passed for input: ['def square(x) { x * x }, 'var g = square', 'g(4)'] Results: 11 passed, 0 failed, 11 total </pre>

Edge Cases & Quality Acceptance Criteria

Scenario / Test Case	Expected Result
Immutability Re-assignment Violation	Attempting to overwrite an established variable mapping throws a descriptive operational error, protecting functional state immutability.
Screenshot: <PUT HERE>	
Division by Zero	Dividing an expression by zero handles the mathematical exception gracefully without bringing down the interactive runtime loop.
Screenshot: <PUT HERE>	 <pre>/Users/admin/CLionProjects/interpreter-anna-mo 7/0 Error: Division by zero </pre>
Syntax & Mismatched Token Errors	Providing malformed expressions (e.g., unmatched parentheses or missing commas in parameters) generates clear recovery logs in the console.
Screenshot: <PUT HERE>	 <pre>/Users/admin/CLionProjects/interpreter-anna-mo min(8 6) Error: expected ')' pow(3 2) Error: expected ')' max(7.3) Error: expects 2 arguments</pre>
Memory Leak Validation	Executing multiple expressions, definitions, and bindings under Valgrind or AddressSanitizer ensures complete dynamic heap cleanups upon exiting.
Screenshot: <PUT HERE>	